

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“JNANA SANGAMA”, BELAGAVI - 590018



DEEP LEARNING MINI PROJECT REPORT(CS722T2C)
on
“SIGNBRIDGE: REAL-TIME SIGN LANGUAGE ALPHABET
RECOGNITION SYSTEM”

Submitted by

ARJUN K V	4SF23CS032
ADITYA MAHAPATRA	4SF23CS010
J ARJUN RAO	4SF23CS078

In partial fulfillment of the requirements for the VII semester

BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Ms. Shiji Abraham

Assistant Professor

Department of CSE

at



SAHYADRI

College of Engineering & Management

An Autonomous Institution

MANGALURU

2026- 27



EVALUATION RUBRICS

Name: Arjun K V

USN:4SF23CS032

Sl. No.	Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	Marks
1	Problem Definition & Objectives	Problem is clearly defined, relevant, and objectives are specific and measurable	Problem and objectives are clear with minor gaps	Problem is partially defined; objectives need improvement	Problem/objectives are unclear	
2	Technical Approach & Implementation	Appropriate methodology, algorithms/tools used effectively; implementation is complete	Appropriate technical approach with minor issues	Basic implementation with limited technical depth	Incomplete or inappropriate implementation	
3	Innovation / Application / Problem Solving	Demonstrates strong originality, creativity, and effective solution to the identified problem	Shows some innovation and good problem-solving	Limited innovation; mostly conventional approach	No significant innovation/problem-solving	
4	Results, Testing & Analysis	Results are clearly presented, thoroughly tested, and critically analysed using suitable measures	Results presented with adequate testing and analysis	Basic results and limited testing/analysis	Results are incomplete or unsupported	
5	Documentation , Presentation & Viva	Well-structured report, clear presentation, excellent communication and confident viva	Good documentation , presentation and satisfactory viva	Adequate documentation/presentation with limited clarity	Poor documentation, presentation or viva	
	Total					



EVALUATION RUBRICS

Name: Aditya Mahapatra

USN:4SF23CS010

Sl. No.	Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	Marks
1	Problem Definition & Objectives	Problem is clearly defined, relevant, and objectives are specific and measurable	Problem and objectives are clear with minor gaps	Problem is partially defined; objectives need improvement	Problem/objectives are unclear	
2	Technical Approach & Implementation	Appropriate methodology, algorithms/tools used effectively; implementation is complete	Appropriate technical approach with minor issues	Basic implementation with limited technical depth	Incomplete or inappropriate implementation	
3	Innovation / Application / Problem Solving	Demonstrates strong originality, creativity, and effective solution to the identified problem	Shows some innovation and good problem-solving	Limited innovation; mostly conventional approach	No significant innovation/problem-solving	
4	Results, Testing & Analysis	Results are clearly presented, thoroughly tested, and critically analysed using suitable measures	Results presented with adequate testing and analysis	Basic results and limited testing/analysis	Results are incomplete or unsupported	
5	Documentation , Presentation & Viva	Well-structured report, clear presentation, excellent communication and confident viva	Good documentation , presentation and satisfactory viva	Adequate documentation/presentation with limited clarity	Poor documentation, presentation or viva	
	Total					



EVALUATION RUBRICS

Name: J Arjun Rao

USN:4SF23CS078

Sl. No.	Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	Marks
1	Problem Definition & Objectives	Problem is clearly defined, relevant, and objectives are specific and measurable	Problem and objectives are clear with minor gaps	Problem is partially defined; objectives need improvement	Problem/objectives are unclear	
2	Technical Approach & Implementation	Appropriate methodology, algorithms/tools used effectively; implementation is complete	Appropriate technical approach with minor issues	Basic implementation with limited technical depth	Incomplete or inappropriate implementation	
3	Innovation / Application / Problem Solving	Demonstrates strong originality, creativity, and effective solution to the identified problem	Shows some innovation and good problem-solving	Limited innovation; mostly conventional approach	No significant innovation/problem-solving	
4	Results, Testing & Analysis	Results are clearly presented, thoroughly tested, and critically analysed using suitable measures	Results presented with adequate testing and analysis	Basic results and limited testing/analysis	Results are incomplete or unsupported	
5	Documentation , Presentation & Viva	Well-structured report, clear presentation, excellent communication and confident viva	Good documentation , presentation and satisfactory viva	Adequate documentation/presentation with limited clarity	Poor documentation, presentation or viva	
	Total					



SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Computer Science & Engineering

DECLARATION

We hereby declare that the entire work embodied in this Mini Project Report titled **SIGNBRIDGE: REAL-TIME SIGN LANGUAGE ALPHABET RECOGNITION SYSTEM** has been carried out by us at Sahyadri College of Engineering & Management, Mangaluru under the supervision of **Ms. Shiji Abraham**, in partial fulfillment of the requirements for the VII semester of **Bachelor of Engineering in Computer Science & Engineering**. This report has not been submitted to this or any other University for the award of any other degree.

Arjun K V (4SF23CS032)

Aditya Mahapatra (4SF23CS010)

J Arjun Rao (4SF23CS078)

Dept. of CSE, SCEM, Mangaluru

Abstract

SignBridge is a real-time American Sign Language (ASL) alphabet recognition prototype that combines computer vision, transfer learning, landmark geometry and a browser-based user interface to convert static hand gestures into readable text. The system uses MediaPipe to locate a single hand and its 21 landmarks from webcam frames, creates a square padded hand crop, resizes the crop to 224×224 pixels, and forwards the preprocessed image to a deep-learning classifier. The main training pipeline uses MobileNetV2 pretrained on ImageNet as a frozen feature extractor followed by global average pooling, dropout and a 29-class softmax classification head. The project repository also contains PyTorch and Hugging Face training alternatives and a runtime inference engine that can select a fine-tuned SigLIP model, a custom PyTorch MobileNetV2 model, or a pretrained Hugging Face model depending on available model files. Landmark-based rules are used to refine predictions for visually similar signs, while temporal majority voting reduces frame-to-frame flicker. A cooldown-based word builder converts stable letters into words and sentences through the Streamlit interface. The dataset is the Kaggle ASL Alphabet dataset containing 87,000 training images across 29 classes: A–Z, space, delete and nothing. The repository documents a 70/15/15 train-validation-test split for the TensorFlow pipeline and evaluation scripts that generate a classification report and confusion matrix. The project report stored in the repository notes that the architecture exceeds 90% validation accuracy within 10 epochs in its documented training workflow; exact test metrics remain run-dependent and are generated by the evaluation script. The resulting system demonstrates a practical software-only approach to assistive communication while preserving local processing of webcam frames.

Table of Contents

Content	Page No.
Abstract	ii
Table of Contents	iii
1. Introduction 1.1 Overview 1.2 Scope & Motivation	1
2. Literature Survey (Refer at least 20 Most recent suitable papers)	2
3. Problem Formulation 3.1 Problem Description 3.2 Problem Statement 3.3 Objectives 3.4 Functional Requirements 3.5 Non-Functional Requirements	4
4. Project Design and Implementation 4.1 Proposed Project Architecture 4.2 High Level Design 4.3 Detailed Design 4.4 Datasets Descriptions 4.5 Tools and Technologies used for Implementation	5
5. Results and Discussion 5.1 Experimentation Environment set up 5.2 Functional Requirements Results achieved (Tabular & Graphical results with Descriptions) 5.3 Non-Functional Requirements Results achieved (Tabular & Graphical results with Descriptions) 5.4 Comparison of Results with Existing works (Cited in Literature Survey) 5.5 Project GUI Snapshots (Major Functionalities) 5.6 Societal Impact of the Project 5.7 SDG Mapped	9
Conclusion and Future scope	14
References	15

1. Introduction

1.1 Overview

Sign language is a visual communication system in which meaning is conveyed through hand configuration, movement, orientation and spatial position. For people who use sign language, communication with non-signers can become difficult when the surrounding community does not have sign-language proficiency. This motivates the development of computer-vision systems capable of recognizing signs without requiring specialized sensor gloves or dedicated hardware. Recent research includes vision-based, wearable-sensor and hybrid recognition systems, with a continuing emphasis on real-time operation, lightweight models and better handling of temporal information.

SignBridge addresses the static alphabet component of this problem. The system is designed around a standard webcam and software-only processing. Instead of passing the complete video frame directly to a classifier, the implementation first locates a hand using MediaPipe and derives a padded square crop from the detected landmark coordinates. This reduces irrelevant background information before classification. The project then applies a lightweight transfer-learning classifier and combines the model output with geometric landmark rules and temporal smoothing. This layered design is intended to improve practical stability during continuous webcam use.

The repository contains a TensorFlow/Keras implementation based on MobileNetV2, a PyTorch GPU training implementation using the same backbone, and a Hugging Face fine-tuning implementation based on a pretrained image-classification model. For deployment, `src/inference.py` checks for a locally fine-tuned Hugging Face model first, then a custom PyTorch model, and finally falls back to a pretrained Hugging Face model. This means the project should be described as a modular recognition system with interchangeable classifier back ends rather than as a single fixed model.

1.2 Scope & Motivation

The scope of the present project is recognition of static ASL alphabet poses and three additional control classes: space, delete and nothing. The system is intended for real-time alphabet spelling rather than complete continuous ASL translation. In particular, the current implementation does not model ASL grammar, facial expressions, two-handed signs as a general linguistic problem, or temporal movement patterns as required for fully dynamic signs. The repository explicitly notes that J and Z are represented as static snapshots in the dataset, although their natural ASL production involves movement.

The motivation is to build an accessible prototype that can run with commodity webcam hardware and

local computation. The project aims to reduce background influence through hand extraction, reduce prediction flicker through temporal voting, and convert stable predictions into words through a simple stateful interface. These choices are consistent with the repository objective of providing an assistive prototype with local processing and no permanent collection of webcam frames.

2. Literature Survey

Recent sign-language recognition research can be grouped into three broad categories: static-image recognition, video-based spatio-temporal recognition, and real-time systems optimized for edge or interactive environments. Vision-only systems avoid the hardware burden of sensor gloves, while hybrid methods increasingly combine CNN features with hand landmarks, recurrent networks, attention mechanisms or transformers. The studies below were selected to provide direct context for the SignBridge design choices.

Study	Approach	Task / Dataset	Contribution / Result
Alsharif et al. (2024)	YOLOv8 with MediaPipe for ASL alphabet recognition	ASL alphabet; 29,820 images	Reported 98% precision, 98% recall, 99% F1 and 98% mAP; demonstrates combined hand tracking and object detection.
Jia and Li (2024)	SLR-YOLO, an improved YOLOv8	ASL Letters and Bengali Sign Language Alphabet	Improved accuracy and reduced model size/FLOPs relative to baseline YOLOv8.
Alaftekin et al. (2024)	YOLOv4-CSP for real-time Turkish sign recognition	Turkish sign language	Reported 98.95% precision, 98.15% recall, 98.55 F1 and 99.49% mAP in 9.8 ms.
Mujeeb et al. (2024)	MediaPipe plus neural networks in a web application	Pakistani Sign Language	Reported 99.8% static test accuracy and 100% dynamic test accuracy in the reported evaluation.
Arya et al. (2024)	YOLO-based lightweight ISL recognition	Indian Sign Language letters and numbers	Reported precision 98.46%, recall 98.78% and mAP@0.5 97.54%.
ZainEldin et al. (2024)	CNN/deep-learning sign-language framework	Sign-language gesture recognition	Explores CNN-based recognition and real-time assistive applications.
Geetha et al. (2024)	YOLOv5-based gesture recognition	Sign-language gestures	Demonstrates object-detection-based sign recognition and translation workflow.
Zhang and Jiang (2024)	Survey of deep-learning sign-language recognition	Multiple sign languages	Reviews CNN, 3D-CNN, YOLO, attention and other deep-learning approaches.
Huang and Chouvatut (2024)	ResNet + LSTM	LSA64 video dataset	Reported 86.25% accuracy, 84.98% F1 and 87.77% precision; highlights temporal modeling requirements.
Noor et al. (2024)	CNN + LSTM hybrid model	Arabic Sign Language	Reported 94.40% CNN and 82.70% LSTM accuracy on their evaluation; separates static and dynamic recognition.

Study	Approach	Task / Dataset	Contribution / Result
Abdul Ameer et al. (2024)	Attention-based deep-learning framework	Arabic Sign Language	Investigates attention mechanisms for improved discriminative feature extraction.
Du et al. (2024)	Granularity-aware fusion and attention	Continuous sign-language recognition	Addresses multiple spatial granularities in continuous sign videos.
Huang et al. (2024)	ViT with MediaPipe angular features	Japanese Sign Language	Reported 99.7% with ViT and 99.3% with CNN in the reported experiment.
Jaiswal et al. (2024)	Hardware acceleration of Tiny YOLOv2	Hindi Indian Sign Language	Reported 32.7 FPS and 61.2% mAP on a Kria KV260 FPGA; studies edge efficiency.
Hugar et al. (2025)	CNN + hand keypoints + LSTM + Transformer	Kannada Sign Language	Combines spatial, structural and temporal features for underrepresented language recognition.
Hasan et al. (2025)	Customized YOLOv8	Sign-language detection	Reports high detection performance and low inference time for a custom sign-language dataset.
Damdoo and Kumar (2025)	SignEdgeLVM transformer	Continuous sign-language translation	Targets edge-device sign-language translation with transformer-based modeling.
Violet and Leena Sri (2025)	Comprehensive survey	Sign-language recognition research 2020–2025	Highlights continued challenges in continuous recognition, dataset diversity and deployment.
Jiang et al. (2026)	Deformable-correlation temporal model	Continuous sign-language recognition	Targets efficient temporal feature modeling while reducing redundant global interactions.
Pose-based SLR study (2026)	MediaPipe landmark representation	Turkish, American and Arabic sign languages	Uses lightweight hand landmarks for cross-lingual static recognition.

The literature suggests that there is no single universally optimal recognition architecture because the requirements differ between isolated static signs, continuous signing, languages, datasets and deployment hardware. SignBridge therefore emphasizes a narrower but practical problem: stable static alphabet recognition from a webcam on commodity hardware. MediaPipe provides an efficient hand representation; MobileNetV2 provides a compact image classifier; landmark rules address specific ambiguities; and temporal smoothing converts frame-wise predictions into stable symbols. This combination places the project between a pure frame classifier and a full temporal translation system.

3. Problem Formulation

3.1 Problem Description

A webcam-based sign-language recognition system must identify the hand pose in the presence of background clutter, camera motion, lighting variation and small changes in the signer’s pose. A direct

frame classifier receives both the hand and the surrounding scene, which increases the opportunity for irrelevant visual features to affect the prediction. In a real-time application, independent frame predictions can also fluctuate between visually similar classes, producing an unstable user experience. Finally, even a correct sequence of letters requires an input mechanism for spacing, deletion and completion of words.

SignBridge treats these difficulties as separate engineering problems. Hand detection isolates the region of interest; the classifier maps a normalized crop to one of 29 classes; a confidence check filters low-certainty predictions; landmark geometry refines ambiguous classifier outputs; temporal majority voting stabilizes the prediction stream; and the word builder converts accepted symbols into readable text.

3.2 Problem Statement

To design and implement a software-only, real-time system that recognizes static American Sign Language alphabet gestures from webcam video using MediaPipe hand extraction and deep-learning image classification, stabilizes predictions using temporal and geometric post-processing, and presents the recognized letters through an interactive word-building interface.

3.3 Objectives

1. Extract the hand region reliably from webcam frames using MediaPipe.
2. Train a lightweight deep-learning classifier for 29 classes consisting of A-Z, space, delete and nothing.
3. Use transfer learning to reduce computational cost and support CPU-oriented real-time inference.
4. Reduce prediction flicker using temporal majority voting over recent predictions.
5. Refine visually similar alphabet predictions using 21-point hand-landmark geometry.
6. Provide a Streamlit interface with live camera control, prediction display, confidence, word construction, deletion and clearing.
7. Evaluate the model with a classification report and confusion matrix and document the limitations of static alphabet recognition.

3.4 Functional Requirements

ID	Functional Requirement
FR1	Load the ASL Alphabet dataset from the configured directory.
FR2	Split the dataset into training, validation and testing

ID	Functional Requirement
	subsets.
FR3	Apply the configured image resizing and augmentation pipeline.
FR4	Train a 29-class deep-learning image classifier.
FR5	Detect a hand and generate a cropped region for inference.
FR6	Predict the sign class and confidence from the cropped hand image.
FR7	Apply landmark-based ambiguity resolution and temporal smoothing.
FR8	Display the recognized sign and confidence in real time.
FR9	Construct words using letters, space, delete and clear operations.
FR10	Generate classification metrics and a confusion matrix for evaluation.

3.5 Non-Functional Requirements

Requirement	Description
Performance	Interactive inference suitable for real-time webcam use; the project report targets at least 15 FPS.
Usability	Simple Streamlit interface with explicit camera and text controls.
Reliability	Stable predictions through confidence filtering, geometric refinement and temporal smoothing.
Portability	Designed for ordinary Python environments and commodity webcams; GPU support is optional.
Privacy	Webcam frames are processed locally and are not intended to be permanently stored or transmitted.
Maintainability	Separation of configuration, preprocessing, model, inference, smoothing and word-building modules.
Scalability	Classifier back ends and future dynamic-sign modules can be replaced or extended.

4. Project Design and Implementation

4.1 Proposed Project Architecture

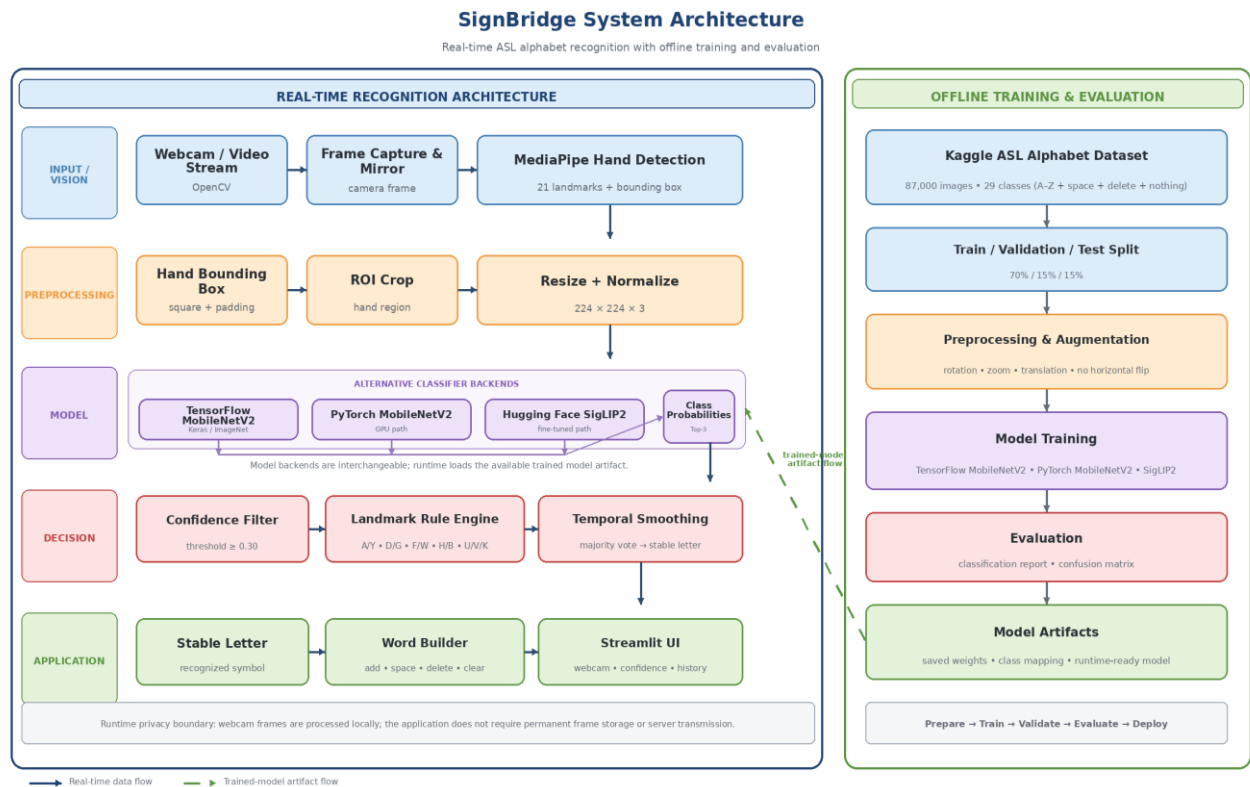


Figure 4.1: Proposed SignBridge system architecture.

The proposed architecture separates the system into an offline training and evaluation pipeline and a real-time inference pipeline. During training, the Kaggle ASL Alphabet dataset is divided into training, validation and test subsets, and the training pipeline supports TensorFlow MobileNetV2, PyTorch MobileNetV2 and a Hugging Face SigLIP2-based classifier. During real-time inference, the webcam frame is processed by MediaPipe HandDetector, which extracts 21 hand landmarks and derives a padded square hand region. The cropped region is resized to 224×224 and passed to the available classifier backend. The prediction is then refined using a 0.30 confidence threshold, landmark-based geometric disambiguation for visually similar alphabet gestures, and temporal majority voting. The stable letter is finally sent to the WordBuilder, which supports letter insertion, spaces, deletion and clearing, and the result is presented through the Streamlit interface.

4.2 High Level Design

Layer	Implementation
Input layer	Webcam BGR frames captured with OpenCV; frames are horizontally flipped for a natural mirror view in the main Streamlit application.
Hand extraction	MediaPipe Tasks API is preferred, with a legacy MediaPipe Hands fallback. One hand is configured for the runtime detector.

Layer	Implementation
Region of interest	Landmarks are converted into a square crop with 25% padding relative to the larger hand dimension.
Preprocessing	Classifier input is resized to 224×224 . TensorFlow MobileNetV2 preprocessing scales inputs for the pretrained backbone.
Classification	Primary documented model: MobileNetV2 pretrained on ImageNet with a frozen base and 29-class softmax head. Alternate PyTorch and Hugging Face routes are also implemented.
Post-processing	Confidence thresholding, landmark disambiguation and temporal smoothing.
Interaction	WordBuilder stores the current word and history, supporting letters, spaces, delete and clear.
Presentation	Streamlit provides the live camera feed and recognition controls.

4.3 Detailed Design

4.3.1 Hand Detection and Cropping

The HandDetector class supports the MediaPipe Tasks API and, if unavailable, the legacy MediaPipe Solutions API. The detector is configured for one hand and a minimum hand-detection confidence of 0.5. When landmarks are available, the implementation converts normalized landmark coordinates into pixel coordinates, computes the minimum and maximum x and y values, and constructs a square bounding box centered on the landmark set. The side length is based on the larger of the raw width or height multiplied by 1.35, which adds contextual padding while maintaining a square aspect ratio.

4.3.2 Classifier and Transfer Learning

The TensorFlow pipeline defines an input size of $224 \times 224 \times 3$ and 29 output classes. MobileNetV2 is loaded without its original classification head and with ImageNet weights. The convolutional base is frozen and followed by global average pooling, dropout with rate 0.5, and a 29-unit softmax layer. The model is compiled with Adam at a learning rate of 0.001 and categorical cross-entropy loss. This is a conventional transfer-learning design in which the pretrained feature extractor provides general visual features and the new classification head learns the ASL-specific mapping.

4.3.3 Data Augmentation

The TensorFlow implementation applies random rotation, random zoom and random translation to the training data. Horizontal flipping is deliberately omitted. The repository explains that flipping can

change the handedness or visual representation of a sign and therefore is not treated as a universally valid augmentation for ASL.

4.3.4 Prediction Selection

The runtime inference engine records the top three candidate classes and their probabilities. The leading prediction and confidence are passed to the landmark rule engine. The rule engine computes rotation-aware finger extension states from the 21 landmarks and contains explicit logic for ambiguities including A/Y, I/Y, A/S/E/M/N/T, D/G/Z, F/W and H/B/U/V/K. If the geometric conditions match a rule, the engine can replace the raw classifier label and raise the associated confidence to a rule-defined floor.

4.3.5 Temporal Smoothing

The TemporalSmoother stores recent predictions in a fixed-length deque. The stable prediction is the most common class in the window when its count reaches at least half of the configured window size. Invalid or no-hand states clear the smoothing history in the Streamlit application. This prevents single-frame classification noise from immediately changing the displayed letter.

4.3.6 Word Construction

The WordBuilder maintains the current word, a history list, a cooldown timer and the last accepted letter. The default cooldown is 1.0 seconds in configuration, while the Streamlit application initializes the builder with a 1.2-second cooldown. The word builder handles normal letters, space and delete operations and can clear the complete current word. The application also provides a hands-free auto-add mode that automatically commits a stable letter when its confidence is at least 0.40.

4.4 Dataset Description

The project uses the Kaggle ASL Alphabet dataset. Kaggle describes the training portion as 87,000 images of size 200×200 pixels divided into 29 classes: the 26 alphabet letters plus SPACE, DELETE and NOTHING. The repository downloads the dataset into `data/raw/asl_alphabet_train/asl_alphabet_train` and validates that 29 class folders are present. The TensorFlow preprocessing module uses a 70/15/15 train-validation-test allocation.

Property	Value / Description
Dataset	Kaggle ASL Alphabet
Training images	87,000

Property	Value / Description
Original image size	200 × 200 pixels
Classes	29
Letter classes	A-Z
Control classes	space, delete, nothing
Training split	70%
Validation split	15%
Test split	15%
Model input size	224 × 224 × 3

4.5 Tools and Technologies used for Implementation

Technology	Use in Project
Python 3.10+	Core programming language
TensorFlow / Keras	Primary transfer-learning training pipeline
MobileNetV2	Lightweight CNN feature extractor and classifier backbone
PyTorch / Torchvision	GPU-oriented alternative training pipeline
Hugging Face Transformers	Optional SigLIP-based fine-tuning path
MediaPipe	Hand detection and 21-point landmark extraction
OpenCV	Webcam acquisition, frame conversion, cropping and visualization
Streamlit	Interactive real-time web UI
NumPy / Pandas	Numerical and data manipulation utilities
Scikit-learn	Classification report and confusion matrix
Matplotlib / Seaborn	Evaluation visualization
Pillow	Image conversion and preprocessing
pytest	Pipeline unit tests

5. Results and Discussion

5.1 Experimentation Environment set up

The repository supports both CPU and GPU execution. The documented TensorFlow model is intended for local or commodity hardware and uses MobileNetV2 for efficient inference. The PyTorch training script automatically selects CUDA when available and enables cuDNN benchmarking on GPU. The Hugging Face script also detects CUDA and supports mixed precision. The project therefore

does not require a fixed vendor-specific GPU to operate, although accelerated hardware can reduce training time.

Parameter	Configured Value
Input size	$224 \times 224 \times 3$
TensorFlow batch size	32
Configured epochs	20 in config; command-line default is 5 in train.py
Learning rate	0.001 for TensorFlow MobileNetV2
Train/Validation/Test	70% / 15% / 15%
Number of classes	29
Confidence threshold	0.30
Smoothing window	6 in configuration
Prediction cooldown	1.0 s in configuration
Runtime UI	Streamlit
Camera interface	OpenCV VideoCapture(0)

There is an intentional distinction between configuration defaults and the command-line defaults of individual scripts. For example, config.py defines EPOCHS = 20, whereas train.py uses a default command-line value of 5 unless another value is supplied. The report therefore treats configuration values as design defaults and avoids claiming a single fixed training duration for all experiments.

5.2 Functional Requirements Results achieved

Req.	Function	Observed implementation result
FR1	Dataset loading	Implemented; repository includes Kaggle dataset setup and class validation.
FR2	Dataset splitting	Implemented; TensorFlow path uses 70/15/15.
FR3	Preprocessing and augmentation	Implemented; 224×224 resizing plus rotation, zoom and translation.
FR4	29-class training	Implemented through TensorFlow MobileNetV2 and alternative PyTorch/Hugging Face scripts.
FR5	Hand detection	Implemented through MediaPipe Tasks/legacy fallback.
FR6	Class prediction	Implemented with top-3 candidate probabilities.
FR7	Stabilization	Implemented through landmark rules and temporal majority voting.
FR8	Real-time display	Implemented in Streamlit with webcam controls and confidence display.

Req.	Function	Observed implementation result
FR9	Word builder	Implemented with letter, space, delete, clear and history.
FR10	Evaluation	Implemented through classification report and confusion matrix generation.

The repository-level project report notes that the architecture reaches more than 90% validation accuracy within 10 epochs in its documented training workflow and describes the prototype as capable of rapid real-time inference on standard CPU hardware. Because the repository does not include the generated `classification_report.txt` or the numeric output from a completed evaluation run, these values are treated as documented project claims rather than independently reproduced measurements in this report.

5.3 Non-Functional Requirements Results achieved

Requirement	Result / Evidence
Performance	Designed for interactive webcam inference; exact FPS depends on hardware and selected back end.
Usability	Streamlit provides Start Camera, Stop Camera, Add Letter, Space, Delete and Clear controls.
Reliability	Temporal smoothing and landmark rules are explicitly implemented to reduce flicker and ambiguity.
Privacy	README states webcam processing occurs locally and frames are not permanently saved or transmitted.
Maintainability	Code is separated into configuration, preprocessing, model, inference, smoothing and word-building modules.
Testing	A pytest pipeline verifies model output shape, temporal smoothing and word-builder behavior.

5.4 Comparison of Results with Existing works

Work	Method	Dataset / Task	Reported result
SignBridge (this project)	MediaPipe + MobileNetV2 + landmark rules + smoothing	ASL Alphabet	Repository reports >90% validation within 10 epochs; exact test metrics are run-dependent.
Alsharif et al. 2024	MediaPipe + YOLOv8	ASL alphabet	98% precision, 98% recall, 99% F1, 98% mAP.
Jia & Li 2024	Improved YOLOv8 (SLR-YOLO)	ASL / Bengali alphabet	90.6% and 98.5% validation accuracy respectively in reported datasets.
Mujeeb et al. 2024	MediaPipe + neural networks	Pakistani SL	99.8% static and 100% dynamic test accuracy reported.
Huang & Chouvatut 2024	ResNet + LSTM	LSA64	86.25% accuracy; 84.98% F1.

Work	Method	Dataset / Task	Reported result
Hugar et al. 2025	CNN + keypoints + LSTM + Transformer	Kannada SL	96.75% validation and 81% test accuracy reported.
Jaiswal et al. 2024	Tiny YOLOv2 + FPGA acceleration	Hindi ISL	32.7 FPS and 61.2% mAP on Kria KV260.

The comparison shows that SignBridge occupies a different point in the design space from pure object detectors and heavy temporal models. Its emphasis is a software-only, interactive alphabet spelling tool that uses hand extraction before classification, while advanced research systems increasingly incorporate temporal modeling for dynamic signs. Direct numeric ranking is not appropriate because the studies use different sign languages, datasets, evaluation protocols and hardware. The relevant engineering comparison is therefore architectural: SignBridge explicitly addresses hand-region isolation, frame stability and word construction in addition to classification.

5.5 Project GUI Snapshots (Major Functionalities)

The repository includes a Streamlit interface rather than a static command-line-only demo. The main interface contains a live camera feed, a recognition panel, a current-word display, sentence history, Start Camera and Stop Camera controls, a Hands-Free Auto Add checkbox, and buttons for adding a letter, inserting a space, deleting the last character and clearing the current word. The application also displays alternative candidate classes and a status message based on confidence.

For the final college submission, insert screenshots captured from the running Streamlit application in this subsection. The appropriate screenshots are: (1) initial application view before camera start, (2) live hand landmark/bounding-box view, (3) predicted letter with confidence, (4) current word after multiple stable predictions, and (5) word builder controls with sentence history. The repository source establishes these UI components, but it does not commit screenshots of the finished interface.

f

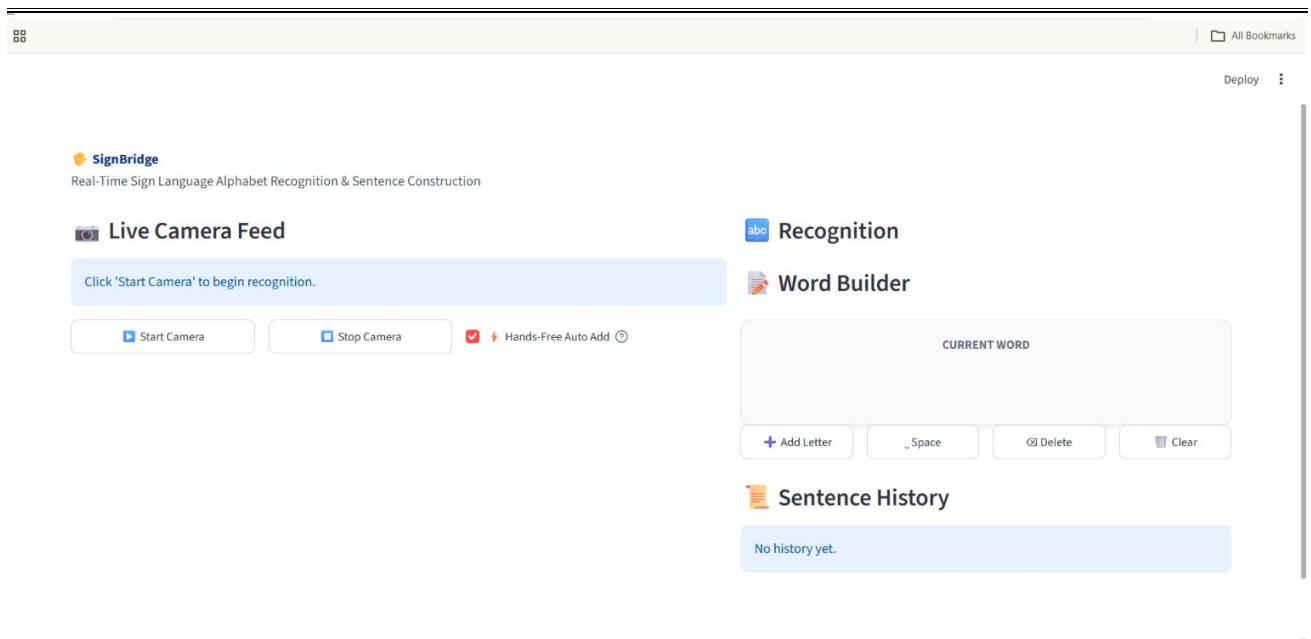


Fig 5.1: Initial SignBridge Streamlit interface

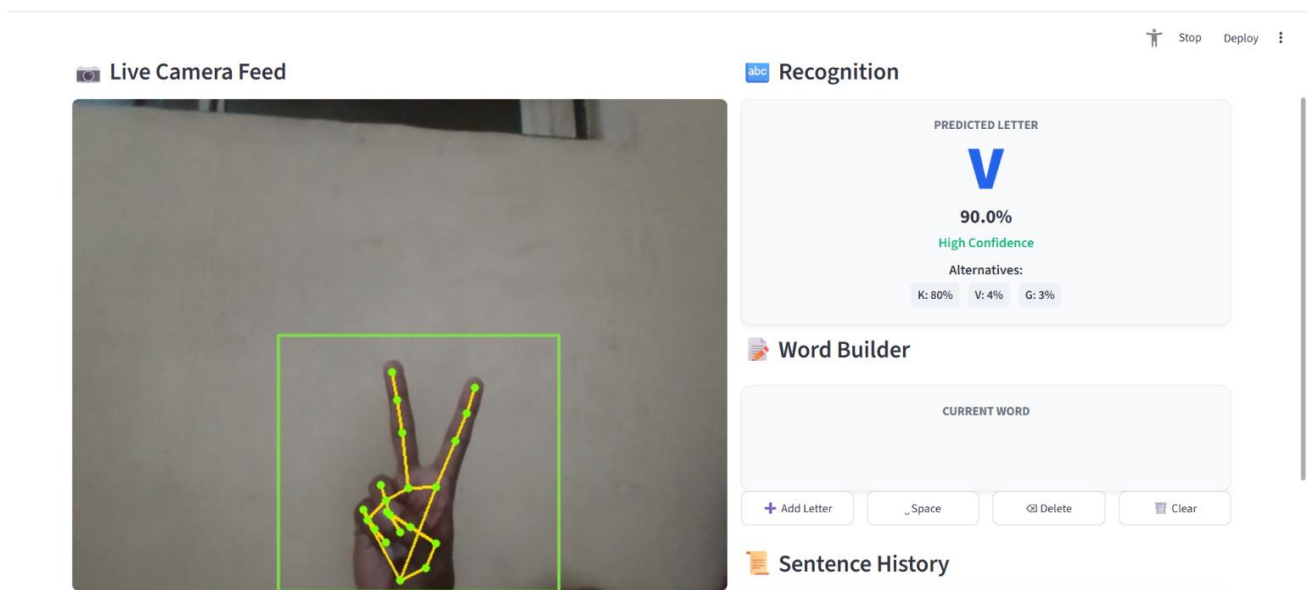


Fig 5.2: Live camera with detected hand and landmark

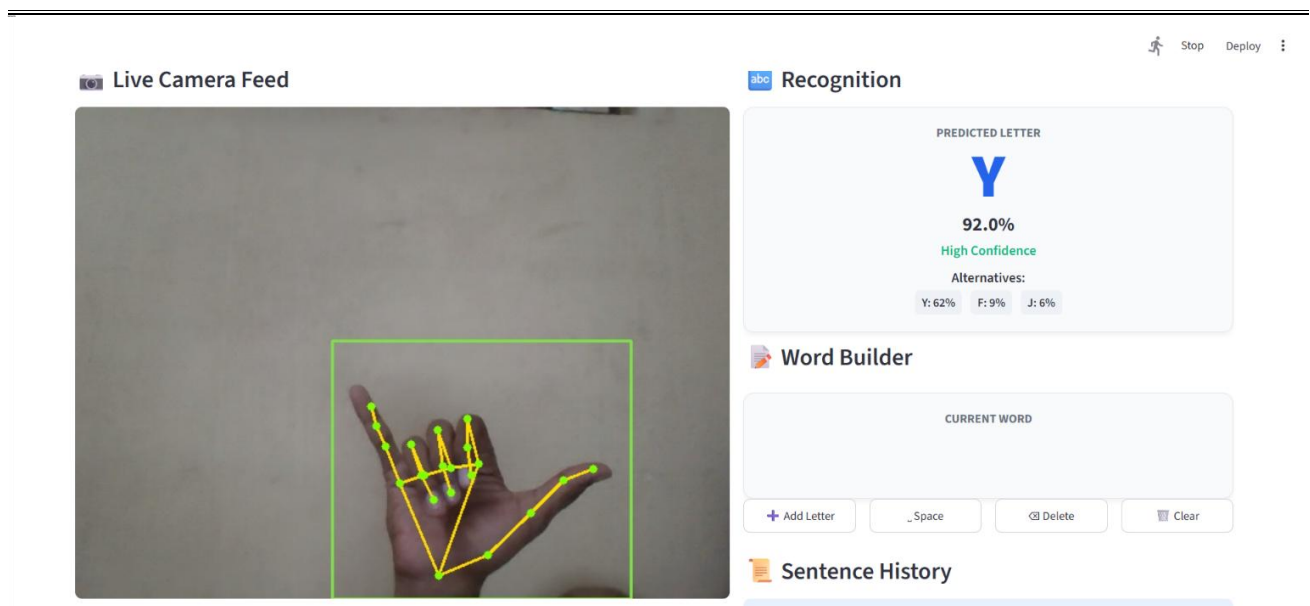


Fig 5.3: Recognition panel with prediction letter and confidence

Suggested Figure	Screenshot to Insert	Purpose
Figure 5.1	Initial SignBridge Streamlit interface	Shows layout and controls.
Figure 5.2	Live camera with detected hand and landmarks	Shows MediaPipe hand extraction.
Figure 5.3	Recognition panel with predicted letter and confidence	Shows classification output.
Figure 5.4	Word Builder after multiple letters	Shows text construction and cooldown behavior.
Figure 5.5	Sentence History and control buttons	Shows interaction and editing features.

5.6 Societal Impact of the Project

SignBridge is intended as an assistive prototype for situations where a signer needs a lightweight software interface to spell alphabetic content to a non-signer. Potential application areas include educational environments, public-service counters, accessibility demonstrations, and human-computer interaction. Because the system uses a webcam rather than wearables, it avoids the acquisition and maintenance burden associated with sensor gloves.

The project also has clear limitations and ethical considerations. Recognition quality depends on the dataset and camera conditions. The Kaggle dataset contains static images, so the natural movement-based production of J and Z is only approximated by static examples. The system should not be

represented as a replacement for professional interpreters or as a complete ASL translation engine. The repository itself warns against using the prototype as the sole communication mechanism in critical medical or legal scenarios.

5.7 SDG Mapped

SDG	Goal	Relevance to SignBridge
SDG 4	Quality Education	Can support interactive learning and practice of ASL alphabet gestures.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates application of computer vision and deep learning to an assistive-technology problem.
SDG 10	Reduced Inequalities	Aims to reduce communication barriers by providing technology-assisted sign recognition.

Conclusion and Future Scope

SignBridge demonstrates a complete prototype pipeline for real-time static ASL alphabet recognition. The project combines MediaPipe hand extraction, lightweight transfer learning, geometric landmark reasoning, temporal smoothing and interactive word construction in a single software system. Its modular design separates data preparation, model training, inference, post-processing and user-interface logic, allowing different classifier back ends to be evaluated without redesigning the entire application.

The main contribution of the project is not a new deep-learning architecture; rather, it is the integration of complementary components around a practical interaction requirement. Hand isolation reduces background dependence, MobileNetV2 reduces inference cost, landmark rules handle selected class ambiguities, temporal smoothing improves visual stability, and the word builder translates stable predictions into usable text. This layered architecture is appropriate for a mini-project demonstration of how deep learning can be embedded inside a real-time application.

Future work should concentrate first on empirical benchmarking of the final chosen classifier using the complete test split and the evaluation script. Additional work can then extend the system from static alphabet spelling to dynamic sign recognition by processing sequences of MediaPipe landmarks with LSTM, GRU or transformer models. A two-hand detector, facial-expression features and language-model-based decoding could further extend the system toward continuous ASL interpretation. Model compression and TensorFlow Lite conversion would also support mobile or edge

deployment.

Another useful extension is systematic robustness testing. The system should be evaluated under controlled variations in lighting, camera distance, skin-tone contrast, background clutter, hand orientation and signer identity. Such testing would provide stronger evidence about real-world generalization than a single aggregate accuracy value. Finally, the word-builder layer could be expanded with speech synthesis and configurable language output while preserving local processing for privacy.

References

- [1] A. Nagaraj, "ASL Alphabet," Kaggle dataset, 2018. DOI: 10.34740/KAGGLE/DSV/29550.
 - [2] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device Real-time Hand Tracking," CV4ARVR, 2020.
 - [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018, pp. 4510–4520. doi:10.1109/CVPR.2018.00474.
 - [4] B. Alsharif, E. Alalwany, and M. Ilyas, "Transfer learning with YOLOV8 for real-time recognition system of American Sign Language Alphabet," Franklin Open, vol. 8, 100165, 2024. doi:10.1016/j.fraope.2024.100165.
 - [5] W. Jia and C. Li, "SLR-YOLO: An improved YOLOv8 network for real-time sign language recognition," Journal of Intelligent & Fuzzy Systems, vol. 46, no. 1, pp. 1663–1680, 2024. doi:10.3233/JIFS-235132.
 - [6] M. Alaftekin, I. Pacal, and K. Cicek, "Real-time sign language recognition based on YOLO algorithm," Neural Computing and Applications, vol. 36, pp. 7609–7624, 2024. doi:10.1007/s00521-024-09503-6.
 - [7] A. A. Mujeeb, A. H. Khan, S. Khalid, M. S. Mirza, and S. J. Khan, "A neural-network based web application on real-time recognition of Pakistani sign language," Engineering Applications of Artificial Intelligence, vol. 135, 108761, 2024. doi:10.1016/j.engappai.2024.108761.
 - [8] C. Arya et al., "A Lightweight Solution for Real-Time Indian Sign Language Recognition," 2024 International Conference on Artificial Intelligence and Emerging Technology, 2024. doi:10.1109/GlobalAISummit62156.2024.10947786.
 - [9] H. ZainEldin et al., "Active convolutional neural networks sign language framework," Artificial Intelligence Review, vol. 57, 162, 2024.
 - [10] M. P. Geetha, S. Swetha, S. Subitsha, and K. V. Visnupriya, "Gesture Based Sign Language Recognition for Specially Challenged Using YOLOv5," 2024 International Conference on Signal Processing and Communications / related proceedings, 2024.
 - [11] Y. Zhang and X. Jiang, "Recent Advances on Deep Learning for Sign Language Recognition," Computer Modeling in Engineering & Sciences, vol. 139, no. 3, 2024. doi:10.32604/cmes.2023.045731.
 - [12] J. Huang and V. Chouvatut, "Video-Based Sign Language Recognition via ResNet and LSTM Network," Journal of Imaging, vol. 10, no. 6, 149, 2024. doi:10.3390/jimaging10060149.
 - [13] T. H. Noor, A. Noor, A. F. Alharbi, A. Faisal, R. Alrashidi, A. S. Alsaedi, G. Alharbi, T. Alsanoosy, and A. Alsaedi, "Real-Time Arabic Sign Language Recognition Using a Hybrid Deep Learning Model," Sensors, vol. 24, no. 11, 3683, 2024. doi:10.3390/s24113683.
 - [14] R. S. Abdul Ameer, M. A. Ahmed, Z. T. Al-Qaysi, M. M. Salih, and M. L. Shuwandy, "Empowering Communication: A Deep Learning Framework for Arabic Sign Language Recognition with an Attention Mechanism," Computers, vol. 13, no. 6, 153, 2024. doi:10.3390/computers13060153.
 - [15] Y. Du, T. Peng, and X. Hu, "Beyond Granularity: Enhancing Continuous Sign Language Recognition with Granularity-Aware Feature Fusion and Attention Optimization," Applied Sciences, vol. 14, no. 19, 8937, 2024. doi:10.3390/app14198937.
 - [16] M. Jaiswal, A. Sharma, and S. Saini, "Hardware acceleration of Tiny YOLO deep neural networks for sign language recognition: A comprehensive performance analysis," VLSI Integration, 2024. doi:10.1016/j.vlsi.2024.102287.
-

-
- [17] G. Hugar, R. M. Kagalkar, and A. Das, "Comparative Study of Hybrid Deep Learning Models for Kannada Sign Language Recognition," *International Journal of Computational Intelligence Systems*, vol. 18, article 191, 2025. doi:10.1007/s44196-025-00922-4.
- [18] M. Hasan et al., "Advancing real-time sign language detection for deaf and hearing-impaired communities: a customized YOLOv8 approach with tailored annotations in computer vision," *BMC Artificial Intelligence*, vol. 1, article 11, 2025. doi:10.1186/s44398-025-00010-9.
- [19] R. Damdoo and P. Kumar, "SignEdgeLVM transformer model for enhanced sign language translation on edge devices," *Discover Computing*, vol. 28, article 15, 2025. doi:10.1007/s10791-025-09509-1.
- [20] I. M. Melanshia Violet and R. Leena Sri, "A comprehensive survey on recent advances and challenges in sign language recognition systems," *Discover Artificial Intelligence*, vol. 5, article 419, 2025. doi:10.1007/s44163-025-00629-7.
- [21] S. Alabdullah and A. A. Alneil, "Revolutionizing sign language recognition for hearing-impaired persons using ensemble of deep learning techniques with fine tuning model," *Journal of King Saud University Computer and Information Sciences*, vol. 38, article 41, 2026.
- [22] Y. Jiang, D. Yang, and C. Chen, "Enhancing Continuous Sign Language Recognition via Spatio-Temporal Multi-Scale Deformable Correlation," *Applied Sciences*, vol. 16, no. 1, 124, 2026. doi:10.3390/app16010124.
- [23] "Pose-Based Static Sign Language Recognition with Deep Learning for Turkish, Arabic, and American Sign Languages," *Sensors*, vol. 26, no. 2, 524, 2026.
- [24] "Optimizing Sign Language Recognition Through a Tailored MobileNet Self-Attention Framework," *Applied Sciences*, vol. 15, no. 23, 12622, 2025.
-